PHASE 2 · BEGINNER

Day 3

Control Flow

2 hrs	4	25	■■
DURATION	CONCEPTS	PRACTICE PROBLEMS	DIFFICULTY

Students learn to make decisions in code using if/elif/else, comparison and logical operators, nested conditionals, and Python's modern match statement.

Session Overview

Goal: By the end of this session, students can write programs that make decisions — branching logic with if/elif/else, combining conditions with logical operators, nesting conditionals, and using match for cleaner multi-branch logic.

Outcome: Students will build fare calculators, eligibility checkers, and delivery-charge logic using real Indian e-commerce and daily-life scenarios.

Timeline

Time	Segment	Details
0:00 – 0:15	Recap & Warm-up	Quick review of Day 2 (variables, types, casting)
0:15 – 0:40	Comparison & if/elif/else	Comparison operators, basic branching
0:40 – 1:00	Logical Operators & Nesting	and / or / not, nested conditionals
1:00 – 1:10	Break	—
1:10 – 1:40	The match Statement	Pattern matching, guard clauses
1:40 – 1:55	Guided Practice	Live-code 2–3 guided examples together
1:55 – 2:00	Wrap-up & Preview	Recap + preview Day 4 (Loops)

> ■ TIP
>
> Control flow is the first time students write code that *branches*. Expect indentation errors — this is normal and a great teaching moment about Python's whitespace-based syntax.

Prerequisites & Preparation

Requirements

Requirement	Notes
Python 3.10+	Required for the match statement
VS Code with Python extension	From Day 1 setup
Completed Day 2	Variables, data types, casting, input/print

Instructor Pre-Session Checklist

■ Verify python3 --version shows 3.10 or higher on the teaching machine

■ Open day03_01_conditionals_operators.py and day03_02_match_statement.py in VS Code

■ Pre-run both files once to confirm no errors before the session

■ Have the Day 2 variables file open for the warm-up recap

Step-by-Step Instruction Guide

STEP 1 — Comparison Operators

What to Say

"Before we can make decisions in code, we need a way to compare values. These comparisons always give us back True or False — just like the booleans we learned about yesterday."

```
wallet_balance = 1500
item_price = 1200
print(wallet_balance > item_price)
# Output: True
```

> ■ WATCH OUT
>
> Common student mix-up: using a single = (assignment) instead of == (comparison) inside an if condition. Point this out early.

STEP 2 — if / elif / else

What to Say

"Now let's actually use these comparisons to make decisions. if runs code only when its condition is True. elif checks another condition if the first failed. else catches everything else."

```
marks = 78

if marks >= 90:
    grade = "A+"
elif marks >= 75:
    grade = "A"
else:
    grade = "C"
print(grade)
# Output: A
```

Common Errors

Error	Cause / Fix
IndentationError	Code block under if/elif/else not indented consistently (use 4 spaces)
Missing colon	Forgetting ':' at the end of the if/elif/else line
Wrong elif order	Conditions checked top-down — more specific conditions must come first

■ DONE WHEN

Done when: students can write a 3-branch if/elif/else that correctly grades a marks value.

STEP 3 — Logical Operators & Nesting

What to Say

"What if we need to check more than one condition at once? That's where and, or, and not come in. And yes — you can put an if statement inside another if statement. That's called nesting."

```
age = 22
has_id_card = True

if age >= 18 and has_id_card:
    print("Arjun can enter the concert.")
# Output: Arjun can enter the concert.
```

■ TIP

Use truth tables on the whiteboard for and/or/not before live-coding — it removes a lot of confusion later.

STEP 4 — The match Statement

What to Say

"Python 3.10 introduced a new way to handle 'check one variable against many possible values' — the match statement. It's often cleaner than a long elif chain."

```
month = "July"

match month:
    case "June" | "July" | "August":
        season = "Monsoon"
    case _:
        season = "Other"
print(season)
# Output: Monsoon
```

Common Errors

Error	Cause / Fix
SyntaxError on match	Running on Python < 3.10 — check interpreter version
Missing wildcard case _	match has no fallback — silently does nothing if no case matches

■ DONE WHEN

Done when: students can rewrite one of their if/elif chains from Step 2 as an equivalent match statement.

Session Wrap-Up

Student Checklist

■ I can use ==, !=, >, <, >=, <= to compare values

■ I can write an if / elif / else chain with multiple branches

■ I can combine conditions with and, or, and not

■ I can nest an if statement inside another if statement

■ I can write a basic match statement with a wildcard case _

■ I can add a guard clause (case ... if ...) to a match case

Key Concepts Recap

Comparison operators produce booleans. if/elif/else branches based on those booleans, checked top to bottom. Logical operators (and/or/not) combine multiple conditions. Nesting lets us layer decisions. match offers a cleaner alternative for checking one variable against many literal values, optionally with guard clauses for extra conditions.

Next Day Preview

Day 4 — Loops: for loops, while loops, range(), break/continue/pass, and the lesser-known else clause on loops. Students will repeat the decision-making skills from today inside repeating blocks of code.

> ■ **WATCH OUT**
>
> Troubleshooting note: if a student's match statement 'does nothing', the most likely cause is a missing case _ wildcard, or comparing against the wrong data type (e.g. matching int 5 against string "5").

Quick Reference Card

Daily Workflow Commands

```
python day03_01_conditionals_operators.py
python day03_02_match_statement.py
python day03_practise_problems_01_conditionals.py
python day03_practise_problems_02_match_mixed.py
```

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + `	Open integrated terminal
Shift + Enter	Run selected line(s) in Python interactive
F5	Run/debug current file
Ctrl/Cmd + /	Toggle line comment

Tool / Syntax Summary

Syntax	Use
if / elif / else	Branch based on a condition
== != > < >= <=	Compare two values, returns bool
and / or / not	Combine or invert boolean conditions
match / case / _	Pattern-match one value against many cases
case X if cond:	Guard clause — extra condition on a case